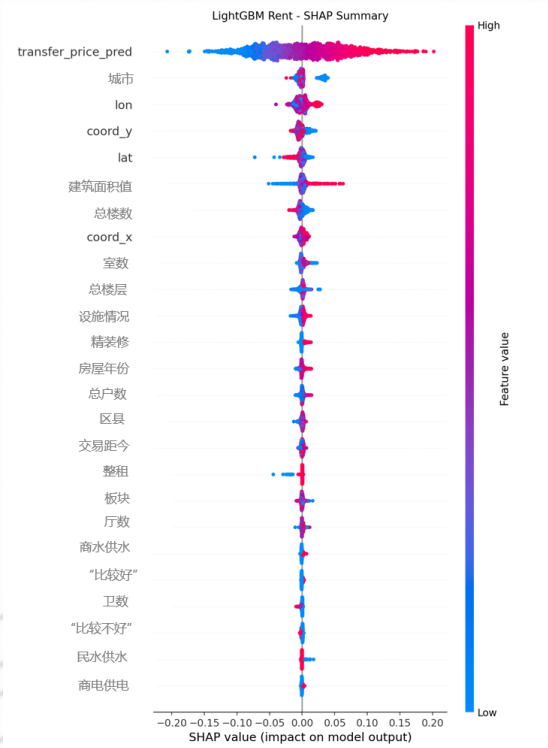
特征处理（半结构化→结构化）

- 统一度量衡：如租期（包含“月/年”单位）、梯户比等
- One-hot处理：租赁方式、建筑结构、供暖方式等
- 文本数据取字符串长度：房屋优势、核心卖点等
- **调用deepseek大语言模型：**客户反馈特征
- 缺失值+异常值处理

SHAP+PDP——特征选择（为未来改进方向做准备）

- 发现**迁移学习变量重要性最高**且特征很干净即价格越高租金越高
  ——房价预测特征可以为租金预测提供额外的信息
- 发现**调用大模型获得的“比较好”等评价对模型有一定预测效果**
  ——但重要性排序没那么高，**后面优先模型的优化**
- **地理位置、房屋时间和房屋基本情况**是影响最大的三大要素



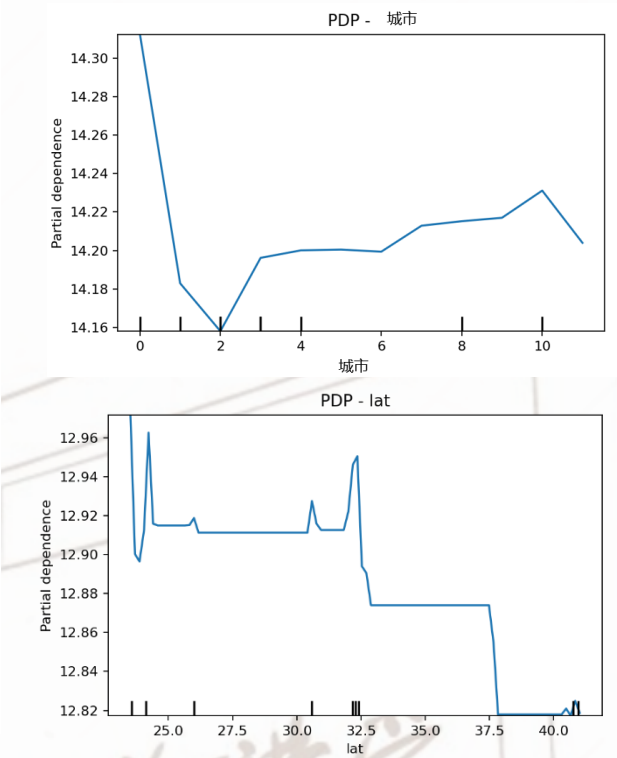


图1 租金预测LGBM模型的SHAP

因变量变化：房价是严重右偏、长尾分布、异方差的

- 尝试1：取ln
- 尝试2：取box-cox变换——ln是其一种特殊情况，可以更好解决房价问题

$$y^{(\lambda)} = \begin{cases} \dfrac{y^{\lambda}-1}{\lambda}, & \lambda \neq 0 \\ \ln(y), & \lambda = 0 \end{cases}$$

Model	Price_in_rmse	Price_oos_rmse	Price_cv_rmse	kaggle
Ridge [noT]+boxcox	0.127297	0.128136	0.127568	37.1
Ridge [noT]+ln	0.481844	0.484590	0.482788	36.2
Ridge [T]+boxcox	0.068723	0.069805	0.068995	65.7
Ridge [T]+ln	0.258676	0.261852	0.259523	66.2
XGBoost [noT]+boxcox	**0.009964**	**0.031141**	**0.030941**	**78.0**
XGBoost [noT]+ln	0.036813	0.116580	0.115948	77.9
XGBoost [T]+boxcox	0.009748	0.030971	0.030775	77.7
XGBoost [T]+ln	0.036500	0.115985	0.115063	77.6

多种模型实验

- 采用了线性模型Ridge、XGboost、RandomForest、Catboost、LightGBM、ANN等多种模型
- **其中XGboost效果最好**

迁移模型

- S1. 找到房价与租金预测的共同特征
- S2. 利用LightGBM模型拟合迁移模型
- S3. 利用房价特征预测rent数值，并将预测值视为房价预测的一个特征
- **最终迁移模型对线性模型有较好作用，对部分树模型有效，但对拟合效果本身较高的模型则有负向作用**

表1 部分模型结果展示

Model	Price_in_rmse	Price_oos_rmse	Price_cv_rmse	kaggle
LightGBM tuned [noT]+boxcox	0.011830	0.032324	0.032133	0.012267
LightGBM tuned [noT]+ln	0.048039	0.120629	0.119554	0.103986
LightGBM tuned [T]+boxcox	0.010891	0.032339	0.031936	0.011685
LightGBM tuned [T]+ln	0.046145	0.119680	0.118691	0.099884
XGBoost [noT]+boxcox	**0.009964**	**0.031141**	**0.030941**	**78.0**
XGBoost [noT]+ln	0.036813	0.116580	0.115948	77.9
XGBoost [T]+boxcox	0.009748	0.030971	0.030775	77.7
XGBoost [T]+ln	0.036500	0.115985	0.115063	77.6
cv_weighted [noT]+ln	/	/	/	**77.8**
cv_weighted [T]+ln	/	/	/	**77.7**
cv_weighted [noT]+boxcox	/	/	/	**77.9**
cv_weighted [T]+boxcox	/	/	/	**77.7**

Optuna调参

- 把 LGBM 的关键超参数纳入 trial 搜索，并用 CV RMSE + early stopping 作为目标函数
- 优化整体运行性能

多模型加权集成：用 CV 表现反推权重

- 谁的 CV 表现更好，谁权重更高
- 采用所有模型进行集成
- **该系列整体分数都较高，模型较为稳定**

全流程 Pipeline 化

- 训练/验证/预测统一接口，降低重复与出错率
- **预处理 → （可选标准化）→ 模型 → split 验证 → CV → final fit → test pred 全部统一起来**

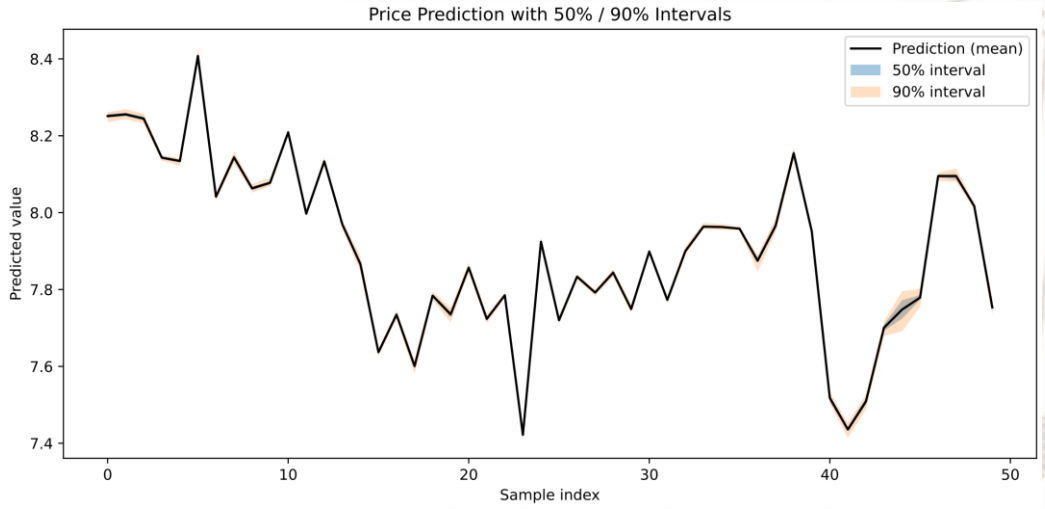
区间预测（Prediction Interval）：

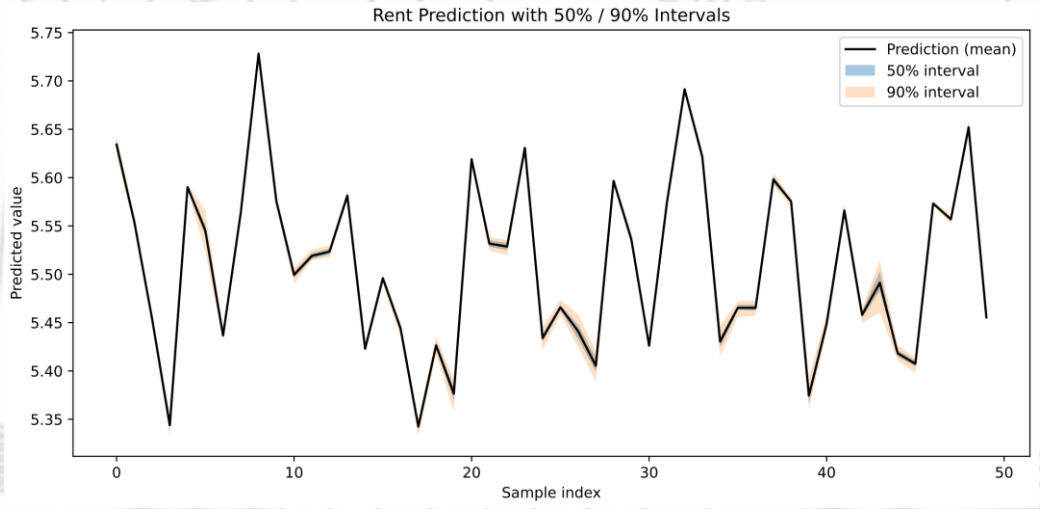
- 一般预测中会给出区间预测结果，以便进行真实情况预估
- Bootstrap RF 给出 50%/90% 区间
- 对训练集做有放回抽样，训练多次 RF，得到测试集预测分布，然后取分位数形成区间（p05/p25/p75/p95 等）

```python
def build_preprocessor(X: pd.DataFrame):
    """
    统一预处理：缺失值处理 + 类别OneHot
    【关键点】统一Pipeline，避免数据泄漏
    """
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
    categorical_cols = [c for c in X.columns if c not in numeric_cols]

    numeric_tf = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
    ])
    categorical_tf = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ])

    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_tf, numeric_cols),
            ("cat", categorical_tf, categorical_cols),
        ],
        remainder="drop"
    )
    return preprocessor, numeric_cols, categorical_cols
```





Model	Price_in_rmse	Price_oos_rmse	Price_cv_rmse	Rent_in_rmse	Rent_oos_rmse	Rent_cv_rmse	kaggle
Ridge [noT]+boxcox	0.127297	0.128136	0.127568	0.059296	0.059695	0.059413	37.1
Ridge [noT]+ln	0.481844	0.484590	0.482788	0.464531	0.467772	0.465418	36.2
Ridge [T]+boxcox	0.068723	0.069805	0.068995	0.037227	0.037410	0.037300	65.7
Ridge [T]+ln	0.258676	0.261852	0.259523	0.284293	0.286519	0.284950	66.2
CatBoost [noT]+boxcox	0.023940	0.032129	0.031927	0.016342	0.021423	0.021509	77.3
CatBoost [noT]+ln	0.088884	0.119662	0.119497	0.122966	0.162923	0.162823	77.2
CatBoost [T]+boxcox	0.023397	0.032267	0.032149	0.016054	0.021407	0.021432	77.3
CatBoost [T]+ln	0.087657	0.120484	0.120411	0.120950	0.162307	0.162061	77.3
LightGBM tuned [noT]+boxcox	0.011830	0.032324	0.032133	0.012267	0.021342	0.021423	77.4
LightGBM tuned [noT]+ln	0.048039	0.120629	0.119554	0.103986	0.162655	0.162170	77.3
LightGBM tuned [T]+boxcox	0.010891	0.032339	0.031936	0.011685	0.021267	0.021340	77.4
LightGBM tuned [T]+ln	0.046145	0.119680	0.118691	0.099884	0.160950	0.160971	77.5
Random Forest [noT]+boxcox	0.016154	0.036227	0.035750	0.010838	0.022696	0.022748	77.2
Random Forest [noT]+ln	0.061045	0.138766	0.135543	0.082264	0.172906	0.172858	77.1
Random Forest [T]+boxcox	0.016816	0.038336	0.037462	0.010848	0.022919	0.023052	76.4
Random Forest [T]+ln	0.063259	0.144615	0.141450	0.082245	0.173878	0.174972	76.3
XGBoost [noT]+boxcox	**0.009964**	**0.031141**	**0.030941**	**0.007831**	**0.021282**	**0.021346**	**78.0**
XGBoost [noT]+ln	0.036813	0.116580	0.115948	0.059935	0.161411	0.161138	**77.9**
XGBoost [T]+boxcox	0.009748	0.030971	0.030775	0.007769	0.021168	0.021193	**77.7**
XGBoost [T]+ln	0.036500	0.115985	0.115063	0.058872	0.159794	0.159954	**77.6**
ANN (MLPRegressor) [noT]+boxcox	0.170265	0.172529	0.153644	0.067896	0.068213	0.096708	-11.3
ANN (MLPRegressor) [noT]+ln	0.533604	0.539292	0.540259	0.532064	0.534634	0.577456	24.6
ANN (MLPRegressor) [T]+boxcox	0.149737	0.151338	0.145756	0.137587	0.137665	0.168177	-61.9
ANN (MLPRegressor) [T]+ln	0.404993	0.407410	0.446130	0.525388	0.528457	0.533677	29.4
ensemble_cv_weighted [noT]+ln	/	/	/	/	/	/	**77.8**
ensemble_cv_weighted [T]+ln	/	/	/	/	/	/	**77.7**
ensemble_cv_weighted [noT]+boxcox	/	/	/	/	/	/	**77.9**
ensemble_cv_weighted [T]+boxcox	/	/	/	/	/	/	**77.7**